

Task1. 字节流和字符流

字节流

1. 设计一个程序 实现功能: 将doro.jpg复制一份到项目根目录下 新图片命名为doro_copy.jpg

```
package com.IOflow;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.File;
public class DoroCopy {
    //将doro.jpg复制一份到项目根目录下 新图片命名为doro_copy.jpg
    public static void main(String[] args) {
        File f1 = new File("doro.jpg");
        File f2 = new File("doro_copy.jpg");

        //先判断源文件是否存在
        if(f1.exists()){
            //try-with-resources 自动关闭流
            try(FileInputStream fis = new FileInputStream(f1);
                FileOutputStream fos = new FileOutputStream(f2)){
                //缓冲区
                byte[] buffer = new byte[1024];
                int len = fis.read(buffer);
                while(len != -1){
                    fos.write(buffer, 0, len);
                }
                System.out.println("复制完成");
            }catch(Exception e){
                e.printStackTrace();
            }
        }else{
            System.out.println("源文件不存在");
        }
    }
}
```

字符流

2. 设计一个程序: 读取name.txt 将表中的人名按照unicode自然排序 每个人名占据一行 在项目根目录下创建name_sorted.txt文件 将排序后的内容写入name_sorted.txt文件 要求去掉多余的首尾空格与空行

```
package com.IOflow;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.FileWriter;
import java.io.BufferedWriter;
import java.io.File;
import java.util.ArrayList;
import java.util.Collections;
```

```

import java.util.List;
import java.io.IOException;
public class NameSort {
    public static void main(String[] args) {
        File f1 = new File("name.txt");
        File f2 = new File("name_sorted.txt");
        //先判断源文件是否存在
        if(!f1.exists()){
            System.out.println("源文件不存在");
            return;
        }
        //创建一个List集合来存储姓名
        List<String> names = new ArrayList<>();
        //try-with-resources 自动关闭流
        //用bufferedReader读取文件内容
        try(BufferedReader br = new BufferedReader(new FileReader(f1))){
            String line;
            while((line = br.readLine()) != null){
                String trimmedLine = line.trim();
                if(!trimmedLine.isEmpty()){
                    names.add(trimmedLine);
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        //对集合进行排序
        Collections.sort(names);
        //将排序后的姓名写入新文件
        try(BufferedWriter bw = new BufferedWriter(new FileWriter(f2))){
            for(String name : names){
                bw.write(name);
                //每个姓名写入一行
                bw.newLine();
            }
            System.out.println("排序完成, 结果已写入" + f2.getAbsolutePath());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

冯十三
 卫十六
 吴十
 周八
 孙七
 张三
 李四
 杨二十
 沈十八
 王五
 王十二
 蒋十七
 褚十五
 赵六
 郑十一

Task2. 序列化和反序列化

```
package com.Serialization;
import java.io.*;
public class Main {
    public static void main(String[] args) {
        Student student = new Student(1, "doro", 2, "114514");
        System.out.println(student);
        try (ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream("student.dat"))) {
            oos.writeObject(student);
        } catch (IOException e) {
            e.printStackTrace();
        }
        Student doro = new Student(null, null, null, null);
        try (ObjectInputStream ois = new ObjectInputStream(new
        FileInputStream("student.dat"))) {
            doro = (Student) ois.readObject();
        } catch (IOException | ClassNotFoundException e) {
            e.printStackTrace();
        }
        System.out.println(doro);
    }
}
```

```
• (base) sterben@wzy:~/Java_structure$ /usr/bin/env /usr/li
de-server/data/User/workspaceStorage/6295bd6701ca427097192
Student{id=1, name='doro', gender=2, phone='114514'}
Student{id=1, name='doro', gender=2, phone='114514'}
❖ (base) sterben@wzy:~/Java_structure$
```

Task3. maven

1. 请你自行上网查阅资料，了解一下什么是maven
 - Maven 是Java 项目的构建与依赖管理工具。
可以帮你自动管理项目、下载类库、编译代码、打包、清理项目
2. 什么是jar包?
 - 把一堆java类打包成一个文件,本质和zip很像
3. maven和jar包有关系吗? 有什么关系?
 - maven就是自动管理jar包的工具,还可以解决冲突和依赖问题
4. 我们为什么要用maven?
 1. 自动管理依赖
 2. 统一项目结构,每个人都能看懂
5. 怎么配置你自己的本地仓库位置?

- 先去 `etc/maven` 路径下找到 `setting.xml` 文件,复制到 `.m2` 文件夹里,修改下图的路径为自己的 **repository**

```
<localRepository>\home\sterben\.m2\repository</localRepository>
```

6. 远程仓库不止一种! 你知道中央仓库是什么吗?

- 是Maven公司自带的仓库,有很多公开的jar包,地址默认写好了

7. 私服又是什么?

- 是公司自己搭建的**Maven服务器**,可以存放公司的**私有jar包**,也可以解决外网下载缓慢的问题

8. 你的IDEA关联maven了吗?

- 我用vscode下载了maven插件

9. 一个maven项目都需要配置什么参数? 你能说说看吗?

1. `groupId`: 组织 / 公司反向域名
2. `artifactId`: 项目 / 模块名字
3. `version`: 项目版本号

10. 你的项目结构是什么样的? 你知道你的代码、配置文件、测试代码、测试配置文件、maven项目配置文件都在哪里吗?

- 业务代码: `src/main/java`
- 项目配置文件: `src/main/resources`
- 测试代码: `src/test/java`
- 测试配置文件: `src/test/resources`
- Maven 项目配置文件: `pom.xml`

11. pom.xml文件?

1. 这是什么?

- 是**项目对象模型**,每个maven项目都要有pom.xml文件

2. 它有什么用?

1. 描述项目信息: 项目名称、版本、归属组织
2. 管理依赖 jar 包: 自动下载、管理 jar 包, 不用手动复制 jar 到项目
3. 管理项目构建: 编译、测试、打包、安装、发布
4. 统一管理版本, 解决 jar 包冲突

3. 它的基本配置有哪些?

- 除了上面提到的 `groupId`, `artifactId`, `version` 还有:
- `packaging`: 打包类型,一般为jar
- `dependencies`: 声明依赖的jar包,里面还可以用 `scope` 声明生效范围
- `properties`: 统一管理版本号

4. 你能通过这个文件管理你的jar包版本、项目类型吗?

- `properties` 统一管理

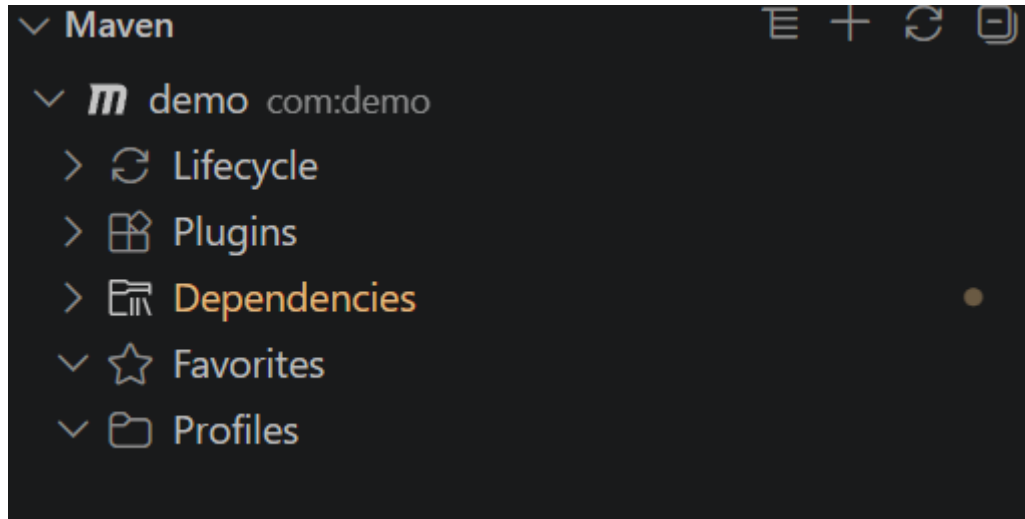
12. 怎么导入你需要的jar包? 去找到答案吧!

- `dependencies`: 声明依赖的jar包,里面还可以用 `scope` 声明生效范围

13. maven的常用命令:

- `mvn clean` # 清理: 删除 target 文件夹 (编译、打包生成的临时文件)
- `mvn compile` # 编译 `src/main/java` 的主代码, 生成class文件到target
- `mvn test` # 执行单元测试 (`src/test/java`下的测试代码)
- `mvn package` # 打包: 先编译+测试, 最后生成 jar/war 包, 放在target
- `mvn install` # 安装: 打包后, 把jar包安装到你的**本地仓库**, 供本机其他项目引用
- `mvn deploy` # 发布: 把jar包上传到远程私服

图形界面:



Task4. 快递取件码查询系统

```
package com;

import com.alibaba.fastjson2.JSON;
import com.sun.net.httpserver.HttpExchange;
import com.sun.net.httpserver.HttpHandler;
import com.sun.net.httpserver.HttpServer;

import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;
import java.nio.charset.StandardCharsets;
import java.util.HashMap;
import java.util.Map;

public class Server {
    private static final int PORT = 8000;
    // 模拟快递柜数据: key = 快递单号_手机号, value = 取件码
    private static final Map<String, String> expressMap = new HashMap<>();

    public static void main(String[] args) throws IOException {
        // 初始化一些测试数据
        initializeExpressData();
        // 创建HTTP服务器, 监听指定端口
        HttpServer server = HttpServer.create(new InetSocketAddress(PORT), 0);
        // 设置路由和处理程序
        server.createContext("/query", new QueryHandler());
        // 启动服务器
        server.start();
        System.out.println("Server started on port " + PORT);
    }

    private static void initializeExpressData() {
        // 添加一些测试数据
        // 键的构成是 快递单号_手机号
        expressMap.put("SF123456789_13005433678", "1234");
        // TODO 1: 参照测试数据表, 把其余3条数据也加进去
        expressMap.put("JD987654321_19805433168", "5678");
        expressMap.put("YT456789123_13905479698", "9012");
    }
}
```

```

        expressMap.put("ZT789123456_18505433664", "3456");
    }

    static class QueryHandler implements Handler {
        @Override
        public void handle(HttpExchange exchange) throws IOException {
            // 非POST请求返回405 Method Not Allowed
            if (!"POST".equals(exchange.getRequestMethod())) {
                exchange.sendResponseHeaders(405, -1);
                exchange.close();
                return;
            }

            try {
                // 读取请求体（请求体是字节流，按UTF-8转成字符串）
                String body = new
String(exchange.getRequestBody().readAllBytes(), StandardCharsets.UTF_8);
                // 解析JSON请求，取出单号和手机号
                QueryRequest request = JSON.parseObject(body,
QueryRequest.class);
                String trackingNumber = request.getTrackingNumber();
                String phone = request.getPhone();

                // TODO 2: 校验单号和手机号不能为空，不合法就抛异常走400分支
                if(trackingNumber == null || phone == null){
                    throw new IllegalArgumentException("单号或手机号不能为空");
                }
                // TODO 3: 用"单号_手机号"拼成key，在expressMap中查询取件码
                String key = trackingNumber + "_" + phone;
                String pickCode = expressMap.get(key);

                // TODO 4: 根据查询结果构造QueryResponse
                QueryResponse response = new QueryResponse();
                if (pickCode != null) {
                    response.setPickCode(pickCode);
                    response.setMsg("success");
                } else {
                    response.setPickCode(null);
                    response.setMsg("手机号或单号不正确");
                }
                // TODO 5: 把响应对象序列化成JSON并发送响应（状态码200）
                String jsonResponse = JSON.toJSONString(response);
                exchange.getResponseHeaders().set("Content-Type",
"application/json; charset=UTF-8");
                exchange.sendResponseHeaders(200,
jsonResponse.getBytes(StandardCharsets.UTF_8).length);
                try (OutputStream os = exchange.getResponseBody()) {
                    os.write(jsonResponse.getBytes(StandardCharsets.UTF_8));
                }
            } catch (Exception e) {
                // 处理异常，返回400状态码(Bad Request)
                // TODO 6: 返回状态码400和错误JSON {"pick_code":null,"msg":"请求格式
错误"}

                exchange.sendResponseHeaders(400, 0);
                QueryResponse errorResponse = new QueryResponse();
                errorResponse.setPickCode(null);
                errorResponse.setMsg("请求格式错误");
                String jsonResponse = JSON.toJSONString(errorResponse);

```

```

        exchange.getResponseHeaders().set("Content-Type",
"application/json; charset=UTF-8");
        try (OutputStream os = exchange.getResponseBody()) {
            os.write(jsonResponse.getBytes(StandardCharsets.UTF_8));
        }
    }
}
}
}
}

```

```

package com;

import com.alibaba.fastjson2.JSON;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.OutputStream;
import java.net.HttpURLConnection;
import java.net.URL;
import java.nio.charset.StandardCharsets;
import java.util.Scanner;

public class Client {
    private static final String SERVER_URL = "http://localhost:8000/query";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        while (true) {
            try {
                // 读取用户输入的快递单号
                System.out.print("请输入快递单号（输入exit退出）：");
                String trackingNumber = scanner.nextLine().trim();
                // TODO 1: 输入exit时退出循环
                if("exit".equals(trackingNumber)){
                    break;
                }
                // 读取用户输入的手机号
                System.out.print("请输入手机号：");
                String phone = scanner.nextLine().trim();

                // TODO 2: 用fastjson构造json格式的请求体，样例为
                // {"trackingNumber":"SF123456789","phone":"13005433678"}
                QueryRequest request = new QueryRequest();
                request.setTrackingNumber(trackingNumber);
                request.setPhone(phone);
                String jsonBody = JSON.toJSONString(request);

                // TODO 3: 创建HttpURLConnection，把上面的json写入请求体并发送HTTP
                POST请求

                // （注意：连接对象要命名为connection，下面判断状态码要用）
                URL url = new URL(SERVER_URL);
                HttpURLConnection connection = (HttpURLConnection)
url.openConnection();
                connection.setRequestMethod("POST");
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    }
}

```

```

        connection.setRequestProperty("Content-Type", "application/json; charset=UTF-8");
        connection.setDoOutput(true);
        try (OutputStream os = connection.getOutputStream()) {
            os.write(jsonBody.getBytes(StandardCharsets.UTF_8));
        }
        // 读取响应, 状态码是200才是响应成功
        if (connection.getResponseCode() == 200) {
            // TODO 4: 读取响应体并解析json, 拿到取件码就打印取件码,
            // 拿不到就打印msg的内容。响应样例:
            // {"pick_code":"1234","msg":"success"}
            // {"pick_code":null,"msg":"手机号不正确"}
            String responseBody = readBody(connection.getInputStream());
            QueryResponse response = JSON.parseObject(responseBody,
                QueryResponse.class);
            if (response.getPickCode() != null) {
                System.out.println("取件码: " + response.getPickCode());
            } else {
                System.out.println("查询失败: " + response.getMsg());
            }
        } else {
            System.out.println("查询失败, 状态码: " +
                connection.getResponseCode());
        }
        connection.disconnect();
    } catch (IOException e) {
        // 请求发生错误(比如Server没启动), 打印错误信息后继续循环
        System.out.println("请求发生错误: " + e.getMessage());
    }
}

scanner.close();
System.out.println("客户端已退出");
}

/** 读取响应体: 把InputStream按UTF-8读成一个字符串 */
private static String readBody(InputStream is) throws IOException {
    StringBuilder sb = new StringBuilder();
    try (BufferedReader reader = new BufferedReader(new
        InputStreamReader(is, StandardCharsets.UTF_8))) {
        String line;
        while ((line = reader.readLine()) != null) {
            sb.append(line);
        }
    }
    return sb.toString();
}

public void sendRequest() {
    // TODO Auto-generated method stub
    throw new UnsupportedOperationException("Unimplemented method 'sendRequest'");
}
}

```

```
package com;
```

```

public class QueryRequest {
//先定义两个属性，快递单号和手机号
    private String trackingNumber;
    private String phone;
    public void setTrackingNumber(String trackingNumber) {
        this.trackingNumber = trackingNumber;
    }
    public void setPhone(String phone) {
        this.phone = phone;
    }
    public String getTrackingNumber() {
        return trackingNumber;
    }
    public String getPhone() {
        return phone;
    }
}

```

```

package com;

public class QueryResponse {
    private String pickCode;
    private String msg;

    public String getPickCode() {
        return pickCode;
    }

    public void setPickCode(String pickCode) {
        this.pickCode = pickCode;
    }

    public String getMsg() {
        return msg;
    }

    public void setMsg(String msg) {
        this.msg = msg;
    }
}

```

```

Server started on port 8000
请输入快递单号（输入exit退出）： JD987654321
请输入手机号： 19805433168
取件码： 5678
请输入快递单号（输入exit退出）： wwdada21
请输入手机号： 12312903
查询失败：手机号或单号不正确
请输入快递单号（输入exit退出）： exit
客户端已退出

```

学习笔记

1. 项目目标:手写客户端和服务端,通过**单号+手机号**查询快递码
2. 技术栈: Java + HttpURLConnection + Fastjson + HTTP/TCP
3. 运行逻辑顺序:
 - 客户端: 输入单号手机号 → 封装 `QueryRequest` 对象 → fastjson 序列化为 JSON
 - HTTP POST, OutputStream 把 JSON 发送给服务端
 - 服务端: 接收请求, 读取 JSON → 反序列化为 `QueryRequest`, 查询模拟快递数据库
 - 封装 `QueryResponse`, 序列化为 JSON 返回客户端
 - 客户端: 读取响应流 JSON → 反序列化 → 判断 `pickCode` 是否存在, 打印**取件码或者错误信息**